

Воспроизведение EAGLE-3 на LLaMA-3.1-8B-Instruct

Ускорение кодом авторов (H200, A100) и в SGLang, скорость обучения: код авторов против NeMo, бюджет GPU-часов

Смена по воспроизведению EAGLE-3

9 октября 2026

Коротко

1. **Статья воспроизводится по т.** Официальный драфт и код авторов дают τ 6,25 против 6,23 в статье ($T = 0$) и 4,91 против 4,92 ($T = 1$); на A100 и в SGLang — то же.
2. **Ускорение зависит от машины:** код авторов на H200 даёт 3,38 $\times$, на A100 — 4,02 $\times$, в статье 4,44 $\times$ ($T = 0$). Ускорение — это отношение к обычной генерации на той же машине: чем она медленнее, тем оно выше. Подробно — в отдельном отчёте про профиль.
3. **SGLang (3b):** 3,06 $\times$ с деревом статьи, но абсолютно в $\approx 1,9$ раза быстрее кода авторов (≈ 555 против ≈ 293 ток/с). **Дерево NeMo по умолчанию (3/1/4) даёт только 1,94 $\times$.**
4. **Обучение (4):** NeMo быстрее кода авторов в 1,06–1,24 $\times$ (каждый в своих настройках), на одинаковой работе — в 1,38 $\times$.
5. **Бюджет (5):** регенерация данных 38 ч + 15–19 ч на эпоху. **В 100 GPU-часов H200 — 2–3 эпохи по всем данным;** 40 эпох, как в конфиге авторов, — 660–800 ч.
6. **В коде авторов три ловушки:** глубина дерева по умолчанию от EAGLE-2, лимит ответа 1024 не передаётся (фактически 512), опубликованный тренер падает при запуске.

Что и как мерили

Tapret — LLaMA-3.1-8B-Instruct (fp16), драфт — официальный yuhuili/EAGLE3-LLaMA3.1-Instruct-8B. Основная машина — одна NVIDIA H200 NVL; для проверки гипотезы про железо 3a повторена на A100-SXM4-80GB (Yandex DataSphere). Код авторов (SafeAILab/EAGLE, коммит cb7e084) — без изменений, вопросы — из их репозитория (5 бенчмарков по 80), формула ускорения — из их speed.py. Дерево как в статье EAGLE-3: 60 токенов, глубина 8, top-k 10. τ — среднее число токенов за один проход большой модели.

Задача	Что	Статус
3a	ускорение кодом авторов (H200; A100 при $T = 0$)	ГОТОВО
3b	то же в SGLang (движок, которым NeMo обслуживает драфты)	ГОТОВО
4	скорость шага обучения: код авторов против NeMo AutoModel	ГОТОВО
5	стоимость регенерации данных и бюджет GPU-часов против 100 ч	ГОТОВО

За. Код авторов: τ совпадает, ускорение зависит от машины

Бенчмарк	T = 0: ускорение / τ			T = 1: ускорение / τ	
	статья	H200	A100	статья	H200
MT-bench	4,40× / 6,13	3,29× / 6,18	3,93× / 6,19	3,07× / 4,24	2,34× / 4,12
HumanEval	4,85× / 6,74	3,74× / 6,74	4,34× / 6,75	4,13× / 5,82	3,17× / 5,81
GSM8K	4,48× / 6,23	3,46× / 6,26	4,06× / 6,28	3,32× / 4,59	2,98× / 4,84
Alpaca	4,82× / 6,70	3,68× / 6,70	4,26× / 6,73	3,90× / 5,56	2,91× / 5,29
CNN/DM	3,65× / 5,34	2,75× / 5,39	3,51× / 5,39	2,99× / 4,39	2,19× / 4,48
Среднее	4,44× / 6,23	3,38× / 6,25	4,02× / 6,27	3,48× / 4,92	2,72× / 4,91

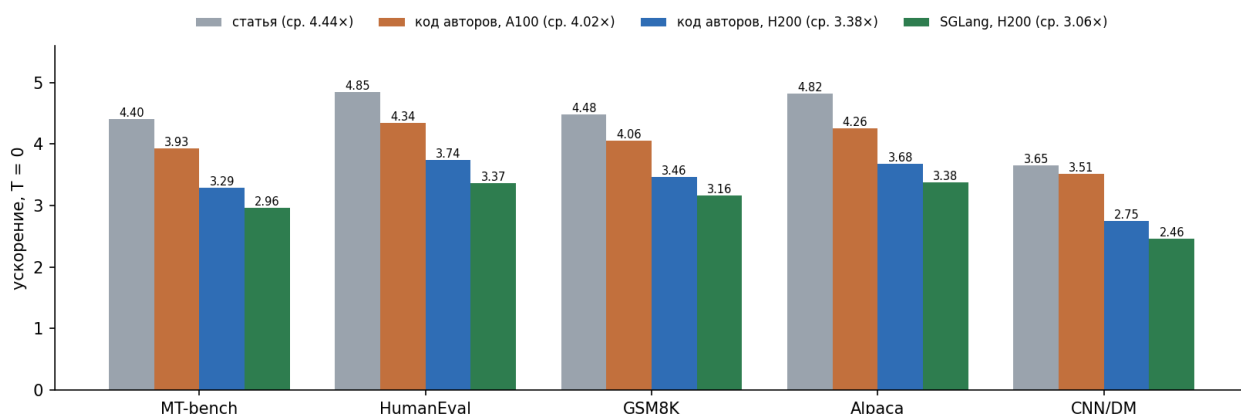


Рис. 1: Ускорение при T = 0 по бенчмаркам: статья, код авторов на A100 и H200, SGLang на H200 (дерево статьи).

Почему ускорение разное при том же τ . Ускорение = τ / (стоимость цикла EAGLE в шагах обычной генерации). Цикл стоит: у авторов $\approx 1,40$ шага, на A100 1,56, на H200 1,85. Профили обеих машин показали:

- по работе GPU цикл стоит $\approx 1,6$ обычного шага на обеих картах — это свойство кода: драфт из 900 мелких ядер плюс проверка дерева, почти целый проход модели;
- по числу запусков ядер — $\approx 1,5$ шага;
- на H200 с быстрым CPU шаг упирается в работу GPU, на A100 в виртуалке DataSphere — в запуск ядер (CPU тратит ≈ 22 мкс на запуск против ≈ 7 мкс на H200). Поэтому на A100 цикл дешевле в обычных шагах (1,47) и ускорение выше.

В абсолютной скорости H200 при этом в 2,8 раза быстрее (≈ 293 против 106 ток/с с EAGLE). Цифра статьи достигается на ≈ 91 % на «медленной» машине; остаток ≈ 10 % объяснить конкретной причиной мы не можем. Подробности и графики профиля — в отчёте «Почему ускорение EAGLE-3 зависит от машины».

Глубина дерева важна. Скрипт оценки авторов по умолчанию строит дерево EAGLE-2 (глубина 6). С ним τ на 10 % ниже статьи (5,55 против 6,23, T = 0), и легко сделать вывод «статья не воспроизводится».

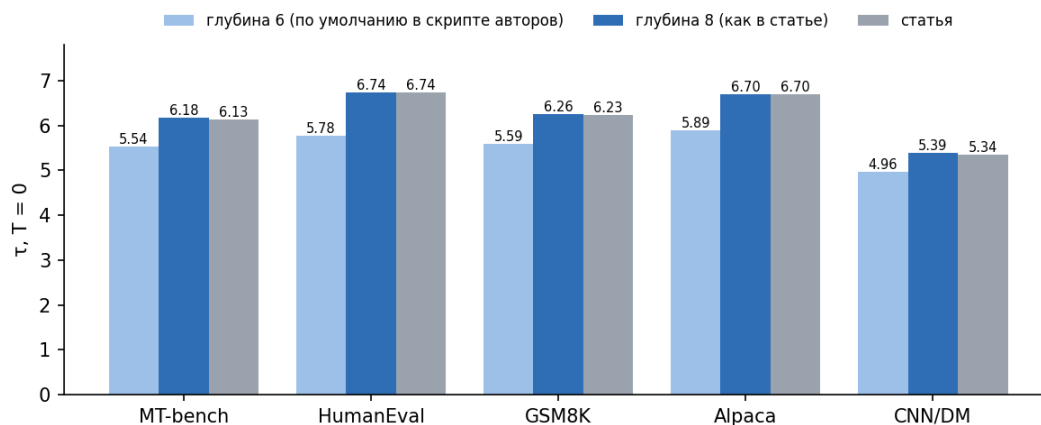


Рис. 2: τ при $T = 0$: глубина по умолчанию в скрипте авторов (6), глубина из статьи (8) и статья.

3b. Тот же драфт в SGLang

SGLang 0.5.9 на той же H200: CUDA graphs, внимание на Triton, без префиксного кэша; вопросы по одному, время — полный HTTP-запрос, не больше 512 новых токенов за ход. Ускорение — относительно обычного сервера SGLang. Деревья — «шаги драфта / top-k / токенов в дереве».

Дерево	Ускорение, $T = 0$	τ , $T = 0$	Ускорение, $T = 1$ (3 seed)
8 / 10 / 60 (как в статье)	3,06×	6,19	2,57× ($\pm \leq 0,03$)
6 / 10 / 60 (EAGLE-2)	2,92×	5,51	—
5 / 8 / 32 (пример из документации SGLang)	2,75×	4,87	—
3 / 1 / 4 (NeMo по умолчанию)	1,94×	2,98	—
код авторов, H200	3,38×	6,25	2,72×

Ток/с ($T = 0$, среднее, приблизительно)	обычная генерация	EAGLE-3
код авторов (HF, eager)	≈ 87	≈ 293
SGLang	≈ 181	≈ 555

3b. SGLang на H200, $T = 0$: шаги драфта / top-k / токенов в дереве

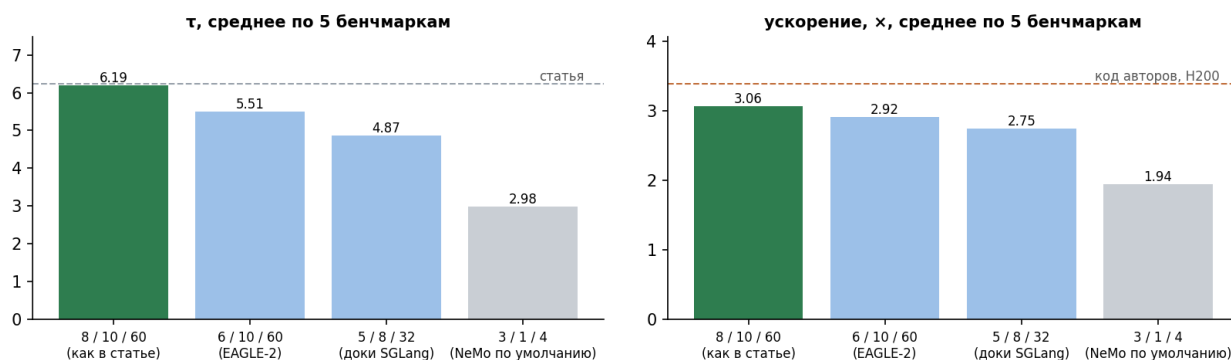


Рис. 3: τ и ускорение в SGLang для четырёх деревьев (среднее по 5 бенчмаркам, $T = 0$).

- τ в SGLang тот же (6,19), драфт работает одинаково.

- **Относительное ускорение ниже, абсолютная скорость выше.** SGLang ускоряет обычную генерацию сильнее (в 2,1 раза), чем EAGLE (в 1,9 раза): отношение к более быстрой базе меньше. Это тот же эффект, что H200 против A100 в 3а. Таймеры разные (CUDA у авторов, HTTP у SGLang), поэтому абсолютные числа сравниваем грубо.
- **Дерево NeMo по умолчанию теряет треть ускорения** (1,94× против 3,06×, τ 2,98 против 6,19). Для реальной отдачи нужно 8 / 10 / 60.
- **T = 1 стабилен по seed:** разброс не больше 0,03×.

Найдено в коде авторов

- **Глубина дерева по умолчанию** в `gen_ea_answer_llama3chat.py` — `--depth 5`, это дерево EAGLE-2. Для EAGLE-3 нужно `--depth 7` (глубина 8 в статье: в коде глубина считается как длина ветки минус 1).
- `--max-new-token 1024` объявлен, но в генерацию не передаётся: фактический лимит ответа — 512 токенов.
- **Опубликованный тренер `traineagle3` падает при запуске:** `cnets.py` читает `self.train_config.gradient_checkpointing`, а `main.py` передаёт словарь. Исправление — две строки; обучить драфт их кодом без него нельзя.
- `speed.py` считает только ускорение (с жёстко прописанным токенизатором LLaMA-2); τ берём по определению статьи из счётчиков генератора.

4. Скорость шага обучения: код авторов против NeMo

Одни и те же 1000 примеров (PerfectBlend, ответы LLaMA-3.1-8B) и общий словарь драфта на 32K; замер по шагам оптимизатора после 20 шагов прогрева.

Запуск	Токенов/с	Относительно авторов
Код авторов как есть (fp16, gradient checkpointing, без паддинга, их расписание lr)	6 925	1,00×
NeMo, упаковка примеров в строки по 2048	7 326	1,06×
NeMo, + <code>torch.compile</code>	8 454	1,22×
NeMo, + <code>torch.compile</code> + FP8	8 577	1,24×
Одинаковая работа (BF16, каждый пример дополнен до 2048): авторы / NeMo	1 638 / 2 259	NeMo 1,38×

4. Скорость шага обучения драфта, одна H200

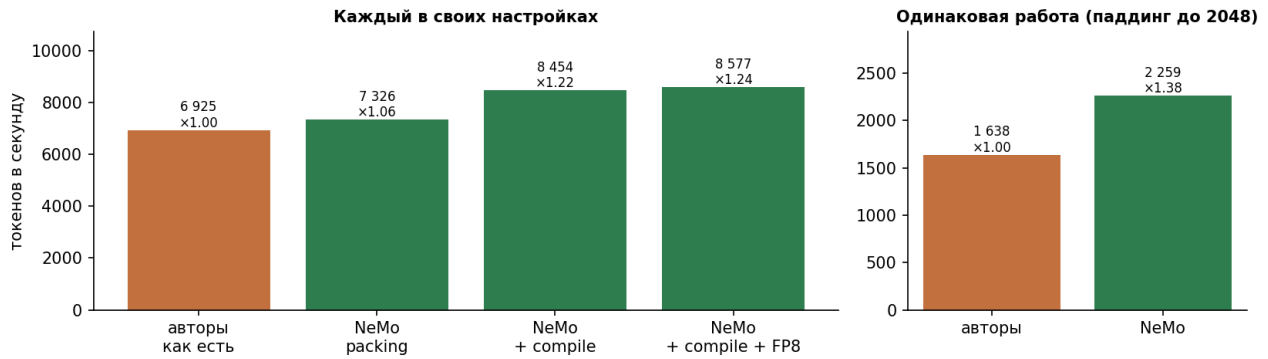


Рис. 4: Скорость шага обучения драфта на одной H200.

- **Выигрыш NeMo скромный — около 1,2×.** Код авторов не тратит время на паddинг (микробатч 1, каждый пример своей длины); главный приём NeMo — упаковка — даёт по сути то же.
- **torch.compile даёт +15 %, FP8 — ещё +1,5 %:** драфт — один слой, его матрицы слишком малы, чтобы FP8 окупился.
- **Память — 28–37 ГБ из 141 ГБ:** запас можно использовать для большего батча.
- **Оговорки.** Сравнивается скорость двух стеков, а не одинаковое обучение: разные версии torch (2.5.1 и 2.10), детали loss, RoPE драфта, точность оптимизатора. flash-attn 2 не проверен.

5. Бюджет GPU-часов

Обучение с нуля, как в статье: сначала таргет переписывает ответы ассистента в обучающих диалогах (регенерация, через обычный сервер SGLang, 64 параллельных запроса), затем эпохи обучения со скоростью из задачи 4. Корпус: ShareGPT V4.3 — 68 623 диалога (это и есть 68K статьи), UltraChat-200K — 208K + 256K; всего ≈532 тыс. Политика регенерации наша (в статье её нет): все ходы ассистента, greedy, не больше 512 токенов на ход и 1900 на пример. 10 ч из 100 — запас на оценку и перезапуски.

	Код авторов	NeMo + compile + FP8
Регенерация (один раз)	38 ч	38 ч
Эпоха	18,9 ч	15,3 ч
Все данные: сколько эпох влезает в 100 ч	2 (86 ч)	3 (94 ч)
5 эпох	68 % данных	79 % данных
40 эпох, как в конфиге авторов	11 % данных; все данные — 804 ч	14 %; все данные — 658 ч

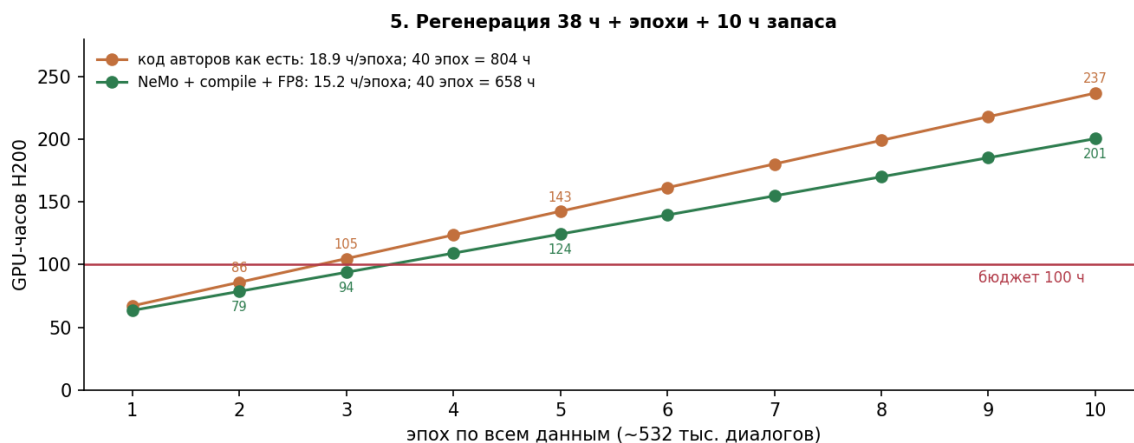


Рис. 5: GPU-часы H200 на обучение по всем данным в зависимости от числа эпох; красная линия — бюджет.

- **Полное воспроизведение (40 эпох) стоит $\approx 660\text{--}800$ GPU-часов H200** — в 6–8 раз больше бюджета.
- **В 100 ч — 2–3 эпохи по всем данным.** Урезать лучше эпохи, а не данные: главный результат EAGLE-3 — рост качества драфта с объёмом данных.
- **Кэшировать скрытые состояния таргета нереально:** 11,6 ТБ.
- **A100 для обучения невыгодна:** по оценке (не замер) в $\approx 2,5$ раза медленнее и без FP8; одна регенерация — ≈ 95 ч.
- **Оговорки.** Регенерация, скорее всего, посчитана с запасом: время мерили группами по 64 диалога вместе с «хвостом», кэш KV был занят на 3 %; большой батч может ускорить её в 2–4 раза (не замерено). Скорость обучения мерили на другом корпусе, доли длин диалогов — по выборке. Разброс $\times 0,75\text{--}1,5$ — проверка чувствительности, а не доверительный интервал.

Что дальше

- **Драфт мы пока не обучали:** стоимость обучения оценена, но не проверена.
- **Замерить то, что может сдвинуть бюджет:** регенерация батчем 256+ непрерывным потоком, профиль шага обучения, большой микро-батч, SDPA-flash. Если оценки подтвердятся, в 100 ч поместится 5–7 эпох вместо 2–3.
- **Пробное обучение** (1–3 эпохи на части данных) с ранней остановкой по τ на отложенной выборке: нужны ли вообще 40 эпох.
- **Для сравнения с другими работами** мерить и ускорение, и абсолютные ток/с: относительное ускорение зависит от машины и движка.

Код, все числа и протокол — в репозитории, папка [reproduction/](#).